Fall 2005 iCTF Report

Metroplex (Omega) Team Members

Terrence Cole
George Vietor Davis
Kevin Hawkins
Michael Helmeste
Andrew Hewatt
Anderson Lam
Bita Mazloom
Dominic Metzger
Ramya Raghavendra
Anders Smestad
Matt Sun
Ricky Taing
Matt Van Gundy
Phil Wittrock

Department of Computer Science
University of California at Santa Barbara
(cs290f-omega@lists.cs.ucsb.edu)

***Summary* – In preparing for the fall 2005 Capture the Flag Competition the members in team Metroplex (Omega) decided to divide into subgroups according to each members task. Before the competition each person signed up for some of the 12 identified different subgroups according to their strengths and interests. Since most members were first time competitors, communication and clarity on role responsibilities and procedures proved to be a difficulty. A larger hindrance to our team's success was as a result of complicated setup and lack of resources. On the other hand, the commitment of our system administrators in keeping the services online helped the team overcome some of its setbacks. In addition, the success of our team lay in each members extensive training in finding vulnerabilities.**

I. Contents Contents

This report is divided into four sections: Initial Team Preparation and Organization, Administration Duties, Attack and Defense Strategies of Services, Thoughts and Ideas for Future iCTF competitions.

II. Initial Team Preperation and Organization

Initial Team Organization

We attempted to organize the team in terms of the active role that they would play during the iCTF rather than assigning responsibilities by the vulnerable services running. We had auditors that were to find vulnerabilities in the different areas of the system, patchers were to patch those vulnerabilities in the running system, and exploiters were to automate exploits to exploit other systems. We also, had system and network admins, human IDS, and Snort signature authors who would develop signatures to filter out known exploits. Job assignments:

Network Administrators were in charge of routing, fire walling and connectivity issues. The persons assigned were: Michael Helmeste and Matt Van Gundy.

System Administrators were in charge of making system level changes to the hosts that where running. The person assigned was: Matt Van Gundy.

Sysconfig Auditors were in charge of auditing the configuration of system software for vulnerabilities. The person's assigned were: Ricky Taing and Bita Mazloom.

Vulnerable Service Auditors were in charge of finding vulnerabilities in the vulnerable services. The persons assigned were: Dominic Metzger, Ricky Taing, Kevin Hawkins, Anderson Lam, Terrence Cole and Bita Mazloom.

Vulnerable Service Patchers were responsible for fixing

vulnerabilities in the vulnerable services found by the auditors. The persons assigned were: Terrence Cole, Phil Wittrock and Matt Sun.

Exploit Writers were responsible for developing exploits for the various vulnerabilities found by the auditors. The person's assigned were: Kevin Hawkins, Anderson Lam and Andrew Hewatt.

IRC PoC was in charge of monitoring the IRC channels. The person assigned was: Anders Smestad.

Human IDS was responsible for watching traffic dumps and logs for exploits in progress and tries to figure out what the scorebot is doing. The persons assigned were Michael Helmeste, Ramya Raghavendra and Vietor Davis.

Reliability PoC was in charge of keeping track of the scorebot. When a service is malfunctioning, they were to let those responsible of the services know. The person responsible was: Ramya Raghavendra.

Exploit Masters were in charge of keeping automated exploits running, figuring out any problems in flag submissions. The person's assigned were: Bita Mazloom, Andrew Hewatt and Matt Sun.

Snort Signature Author developed signatures in order to drop malicious traffic. The persons assigned were Vietor Davis and Michael Helmeste.

Finally we democratically voted on two tyrants who were Dominic Metzger and Bita Mazloom.

Team Perpetration

Dominic created a Wiki page with strategies, thoughts, tips, tricks, list of preparations for iCTF, facts about & HOWTOs for iCTF. In addition he created a submission script that kept log files of errors, flags submitted (so that the same flag would not be submitted twice), unexpected errors, and of flags that were not valid (to stop out command injections through flags). Furthermore, we created a flag-submission-service to test the script. Unfortunately since we got our automated exploits to work very late in the competition, all the log files were overkill and a much simpler script would have been enough. Also, he prepared a toolbox of existing tools for vulnerabilities and exploits.

III. Adminstration Duties

Network Setup

Matt V. and Michael partitioned the network (figure 1) into several different subnets which were protected from one another by firewalls. We placed a "snort box" inline in between the team box and the image box in order to drop any connections that matched signatures which we determined indicated malicious attacks. The team box firewall was relatively simple... it NATed connections into the machines from the CS network and limited traffic out to DNS and the UCSB FTP server (for packages). The Image Box had a rather extensive firewall that prohibited almost all traffic to the host itself. It also had extensive rules for the various images running on it in order to keep a compromise of the vulnerable image from harming any of the other hosts on our subnet.

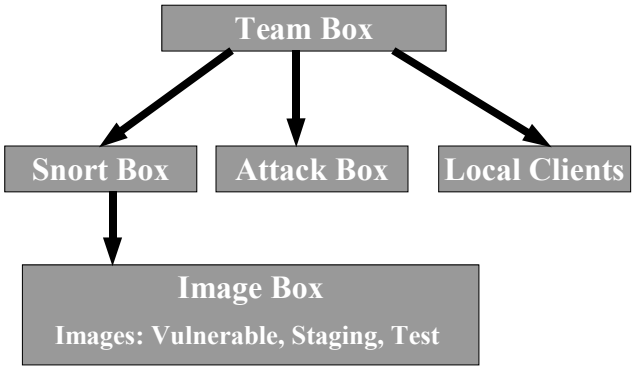


Figure 1: Network setup

Success of Network Setup

Initially, we planned to run the following images:

* Vulnerable image - subject to attacks by remote teams

* Staging image - a clone of the vulnerable image were patches are staged (also used for fail-over if someone rm -rf /'s our vulnerable image)

* Unpatched image - an image not changed since distribution to facilitate testing of our attacks on an unchanged host

* Devel image - an image used for developing exploits and patches that would have a toolchain (as opposed to vulnerable and staging images)

Unfortunately, we found out on the morning of the iCTF that our host didn't have sufficient resources to run all the images planned simultaneously, we had tested them separately. So, we had a vulnerable image, a staging image (which didn't get used effectively) and we used our attack box as a basis for developing exploits.

We had also planned to version control all configuration files on the hosts and all patches to source code. However, during the time of the competition, these procedures proved too cumbersome due to the lack of familiarity with them and not having a chance to iron out the kinks beforehand.

Duties of Tyrants

To increase team efficiency and communication Dominic and Bita were chosen as the acting tyrants. The tyrants duties were to organize the different sub teams so that everyone knew what to do and check that each person's progress. Other duties were, being the point of contact for problems and concerns, helping team members find

solutions to their problems.

Defense Strategies

Our defense strategy was basically,

1. Firewall aggressively to prevent as much network abuse as possible

2. Attempt to patch vulnerabilities that could be easily fixed

3. Using Snort Inline running on the filter box to trigger actions based on the content of a connection (such as dropping the connection).

4. Lock down access to the vulnerable image to prevent connections to insecure accounts.

Difficulties Encountered in Defense

Vietor and Matt V.'s job was to apply our defense strategies 1,3 and 4 during the game. During the game we faced numerous difficulties:

1. Our firewalling was too aggressive for the small amount of testing it received.

2. Snort filtering was very effective, except that, eventually some of the rules caused false positives and filtered out the scorebot.

3. We had also planned to run a module-less kernel with security patches and chroot all the services possible but were short on administration muscle, it proved hard enough to determine the causes of problems with the vulnerable image we had running.

4. Format of the score bot requests changed mid game. As it occurred during a period of other difficulties it was not caught for a while. This proved very problematic.

The large number of problems with the infrastructure that occurred in the first half of the games made it very difficult to establish a baseline of sane activity on which to model defense.

IV. Attack and Defense Strategies of Services

Exploit Writers

Phil and Andrew wrote several scripts to automatically submit flags. One script ran an exploit on every machine and submitted each flag printed out by the exploit. The second script was similar with the difference that it ran every exploit bellow a configurable directory. Both used a third script which would submit the flags.

The scripts were complicated to use because they required a specific output for flags and configuration files for each exploit. Only the first exploit which was an automation template was successful in performing mass attacks on a few of the services. Another difficulty was that the scripts were written during the beginning of the competition in the perl language. Since most people were uncomfortable with perl, a substantial amount of time was wasted in understanding and altering the script to conform to the specifics of applying a services exploit. As a result, we were unsuccessful in gaining points of the "low hanging fruit" and most people ended up exploiting each machine and submit the retrieved flags.

Attack Strategy of Service: ftpd

Terrence analyzed and exploited the vulnerabilities existing in the ftpd service. Ftpd contained many vulnerabilities, in particular information disclosure and buffer overflows. This exploit was by buffer overflows resulting from printf function which put two 512 byte strings into a single 512 byte string. The directory attack occurred because the CWD command did not filter "..". This vulnerability allowed us to freely read flags from most other services. Since this was discovered toward the end of the competition we did not have time to script our attack adequately. If we had spent more time in the first part of the competition in finding the location of the flags for the various services, this could have been an easy method to gain an advantage over our opponents.

Attack Strategy of Service: contribute

Bita and Ricky analyzed and exploited the vulnerability in the contribute service. This service had an SQL injection vulnerability in the history.py script. A select statements which does not sanitize user input allowed us to pass in " or 1=1 – as the value for the id. An example exploit is:

http://localhost:8080/~contribute/cgi-bin/history.py?id=%27%27+or+1% 3D1+--

The result of this exploit is a dump of the database which contains the flags. The last flag which was the most recent was submitted. We changed Andrew's perl script and successfully automated all the exploit. In this way, the script looped through all the servers, does a wget, and parses the output looking form the most recent flag. Unfortunately, no points were gained since we were rather late in submitting this exploit.

Attack Strategy of Service: acquire

Ramya and Phil analyzed and exploited the vulnerability in the acquire service. The PHP code paid_status.php was checking and extracting the status with a get method as follows:

```
<?php
$status=$_GET['status'];
print('<div align="center">');
if ($status == "paid")
{ …
```

However the status variable was a global variable that the user could set in the URL. Since this variable is not being sanitized we were able to exploit this vulnerability by setting "status=paid". As a result, the program would then print out the file that contains the flags. The exploit was automated as follows:

```
wget http://$machines[$i]/~acquire/cgi-
bin/paid_service.php?status=paid
```

Attack Strategy of Service: museum

Kevin analyzed and exploited the vulnerability in the museum service. The vulnerability was by command injection in museum_search.pl. The exploit was a two step process: First, we used "|" to escape the system() execution in order to cat the contents of every file in museum_ideas directory and then we wrote them to a temp file in museum_spam directory. Next, we searched for .* using museum_search.pl which did cat on all files in museum_spam to the webpage (including flags that were copied over by the first step).

Attack Strategy of Service: spam in/out

Anders analyzed and exploited the vulnerability in the spam in and spam out services. If the text SPAM-IN-SPAM-OUT [newline] [[path]filename] [newline] [content] was sent to port 8008, any file can be written anywhere on the system (that the spam user has access to). The service closed the connection with no response. If the text SPAM-IN-SPAM-OUT [newline] [[path]filename] was sent to port 8009, the last line of any file (that the spam user has access to) was sent back to the client. Flags were set in the / home/spam/bin directory of the spam user. In order to read all the flags of a user, they must pass in * for the filename. This returned the last line of the binary file spamin and spamout, so 'binary' response needs to be filtered out. The rest is flags set by the scorebot. It was however not known which was the last flag.

The exploit to write the contents to any file is:

```
perl -e 'print "SPAM-IN-SPAM-
OUT\nfilename\nfile_content\n";' | nc ipaddress
8008
```

The exploit to read the last line of any file is:

```
perl -e 'print "SPAM-IN-SPAM-
OUT\nfilename\n";' | nc 10.20.2.6 8009
```

To prevent attacks we fixed the source code and in the mean time we created a filter that filters out "outbreaks" in the filename. Therefore the only valid strings passed in should consist of [a-zA-Z0-9]. We put the following in a host loop which filters the response for the binary output by:

```
perl -e 'print "SPAM-IN-SPAM-OUT\n*\n";' | nc
$host 8009
```

Since the spamout service was listening to port 8009, it was mistakenly identified to be the tomcat servlet container connection. This caused all outbound traffic to these services to be denied. Later in the competition this was discovered to be a mistake. A firewall filtering rule was installed to deny directory traversal out of the allowed / home/spam/bin path. It appeared that the scorebot used flags containing only alphanumeric characters, so all traffic containing anything else on the second line was dropped.

Attack Strategy of Service: spamback

Anders analyzed the vulnerability in the spamback service. Only the binary to this service was provided for us to analyze. The spamback service was listening on port 2323. The string output of the file contained the "/bin/sh", and a request for a password. This seemed a good place to look for a back door into the system. This service was discovered in the last few minutes of the competition. Hence we did not have enough time to exploit it. We were able to connect to the service, but were not able to get any useful output. The binary contained a lot of text strings that looked like public pgp files. It might have been possible to find the private keys for this service and login without a password.

Attack Strategy of Service: spamgen

Anderson analyzed the vulnerability in the spamgen service. Looking at spamagen.c it seemed like myhash.py had a vulnerability if you could control the string. However, due to lack of time and experience we could not figure out a way to exploit it. The exploit was to print the information in a specific file. The problem we had was in trying to figure out how the hashing worked. This is necessary since the line it returns is the file that gets displayed.

Attack Strategy of Service: spamagent

Ramya analyzed the vulnerability in the spamagent service. The vulnerability was a call to system(dircommand) where dircommand was a concatenation of two user input parameters, username and password. XML command injection was done to exploit this and copy the flags which were in the directories.

V. Thoughts and Ideas for Future iCTF Competitions

In the following competitions we hope to increase our success by doing some things differently. First, we would spend more up-front prep time. There were a lot of ideas that we wanted to implement that we did not have time for (for example a hardened kernel, an effective stating/failover step). Second, next time we want to fully troubleshoot the network setup before the competition begins. Not having all the kinks ironed out proved to be disastrous for the first few hours. Third, when we get the image, we need to define procedures for each role. For example, identify who does exactly what; which person to notify with their findings (this

role was somewhat taken care of by the tyrants); also, to help us find out the different services which were running at different points of the competition we need a procedural checklist of things to look for (this was applied toward the end of the competition).

We lacked experience in simultaneously dealing with maintaining network uptime, finding vulnerabilities, defending our system and exploiting. Although practicing finding vulnerabilities was very helpful, managing all the different activities of the competition proved a whole different ball game. In the future, a test iCTF would be extremely beneficial and would allow the members to work together more smoothly and use the team's system more effectively.

ACKNOWLEDGMENTS

The members of team Metroplex (Omega) would like to thank professor Giovanni Vigna, Marco Cova, Viki Felmetsger, Greg Banks and everybody who spent many waking hours in creating a fun, challenging and memorable competition for us.